Laboratório Prolog · Aula 13

Daniel Mitsuo Tsuyama · RA 24160006-2 · Turma ESOFT6SN-B
Gerado em 20/10/2026, 20:13:52 · tempo registrado no cronômetro: 0:00

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?

Previsão: paradigma(L, funcional) dá L = scheme. paradigma(java, logico) dá false.

Saída do ola.pl

```
?- linguagem(A).
   linguagem(prolog)
   linguagem(scheme)
   linguagem(java)
?- paradigma(prolog,A).
   paradigma(prolog,logico)
?- paradigma(A,funcional).
   paradigma(scheme,funcional)
?- paradigma(java,logico).
   false.
```

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?

paradigma(prolog, P) tem uma variável e pergunta “qual é o paradigma do prolog?” (resposta: P = logico). paradigma(prolog, logico) não tem variável e pergunta “prolog é lógico?” (resposta: true).

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.

Quer dizer que o programa não sabe isso. O fato não está na base, e o Prolog assume que o que não consegue provar é falso (mundo fechado). Ele não afirma nada sobre Java no mundo real.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?

mulher(Quem) tem 7 respostas: lucia, marta, tereza, joana, bia, ana, dora. (mae(M,F), mulher(F)) tem 4 pares: (lucia,tereza), (marta,joana), (tereza,bia), (joana,ana).

Saída do familia.pl

```
mae(joana,jake) → true | mae(jake,joana) → false | pai(maria,jake) → false
mae(joana,A)  → jake, ana
pai(A,jake)   → jorge
mulher(A)     → 7 respostas
(mae(joana,X), homem(X)) → X = jake
(pai(vern,Y), mulher(Y)) → Y = joana
(mae(_,F), mulher(F))    → tereza, joana, bia, ana
```

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

Quais pares (Mae, Filha) existem tais que Mae é mãe de Filha e Filha é mulher?” Teve 4 respostas.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

É a hipótese do mundo fechado: o que não está na base é considerado falso. Na vida real isso é um problema porque “não sei” vira “não”. Por exemplo, uma pessoa que não foi cadastrada seria tratada como alguém sem antecedentes, ou um exame não registrado seria tratado como não realizado.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

A ordem foi lucia, marta, tereza, joana, bia, ana, dora. É a ordem em que os fatos foram escritos no arquivo, porque o Prolog percorre a base de cima para baixo.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake) dá X = paulo. primo(leo, X) dá jake e ana.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

Das quatro unificações, três falham: f(X,X)=f(a,b), [A|B]=[] e 2+3=5. Só f(X,X)=f(a,Y) unifica, com X=a e Y=a.

Saídas de regras.pl e unificacao.pl

```
avo(_, jake)       → marta, vern
avo(osvaldo, _)    → joana, paulo, bia
irmao(jake, _)     → ana
tio(_, jake)       → paulo
primo(leo, _)      → jake, ana
ancestral(osvaldo,_) → vern, tereza, joana, paulo, jake, ana, leo, bia
ancestral(_, jake) → joana, jorge, lucia, marta, osvaldo, vern
ancestral(jake, _) → false
```

```
a = a                       unifica
a = b                       NAO unifica
A = a                       unifica (A = a)
f(A,b) = f(a,B)             unifica (A = a, B = b)
f(A,A) = f(a,b)            NAO unifica
f(A,A) = f(a,B)            unifica (A = a, B = a)
pai(A,jake) = pai(jorge,B)  unifica (A = jorge, B = jake)
[A|B] = [1,2,3]            unifica (A = 1, B = [2,3])
[A|B] = []                 NAO unifica
[A,B|C] = [a,b,c,d]       unifica (A = a, B = b, C = [c,d])
2+3 = 5                    NAO unifica
```

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

A cláusula ancestral(X,Y) :- progenitor(X,Y) é o caso base (pai ou mãe direto). A cláusula ancestral(X,Y) :- progenitor(X,Z), ancestral(Z,Y) é o caso recursivo. Sem o caso recursivo, só se acharia pai e mãe diretos. Sem o caso base, a recursão nunca teria onde parar. Duas cláusulas com a mesma cabeça funcionam como “ou”.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, irmao(jake,_) passa a dar jake e ana. Jake aparece como irmão de si mesmo, porque tem a mesma mãe e o mesmo pai que ele próprio. A meta X \== Y é necessária para excluir esse caso.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Na primeira consulta, a variável X precisaria valer a e b ao mesmo tempo, o que é impossível. Na segunda, X recebe a, e o segundo argumento exige X = Y, então Y também vale a. Ficam X = a e Y = a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Em Java ou Scheme, uma função tem entradas e saída fixas. Uma avo(x) devolveria só os avós, e para achar os netos seria preciso escrever outra função. Em Prolog, avo(X,Z) é uma relação. Qualquer argumento pode ser conhecido ou variável, e a mesma definição responde as duas perguntas.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

concatena(X, Y, [1,2,3]) dá 4 respostas: []+[1,2,3], [1]+[2,3], [1,2]+[3] e [1,2,3]+[].

Saída do listas.pl

```
membro(b,[a,b,c]) → true       membro(z,[a,b,c]) → false
membro(_,[a,b,c]) → a, b, c
tamanho([a,b,c],_) → 3         tamanho([],_) → 0
concatena([1,2],[3,4],_)  → [1,2,3,4]
concatena(_,[3],[1,2,3])  → [1,2]
concatena([1],_,[1,2,3])  → [2,3]
concatena(_,_,[1,2,3])    → 4 respostas
inverte([1,2,3],_)        → [3,2,1]
```

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

O caso base é tamanho([], 0). O passo recursivo é tamanho([_|Cauda],N) :- tamanho(Cauda,M), N is M+1. Em tamanho([a,b,c],N), cada chamada fica esperando a cauda responder para depois fazer o +1. Ficam três somas pendentes, que são resolvidas na volta: 0, depois 1, depois 2 e por fim 3.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Uma lista de 3 elementos pode ser cortada em 4 pontos (com 0, 1, 2 ou 3 elementos na primeira parte), e o backtracking encontra todos. O anexar do Scheme só vai das duas listas para a lista resultante. Ele não consegue, dada a lista final, descobrir quais listas a formam.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

Foi o backtracking junto com a unificação. A primeira cláusula unifica X com a cabeça (a). Ao pedir outra resposta, o Prolog volta e tenta a segunda cláusula, que procura na cauda (b, depois c). Com a lista vazia nenhuma cláusula unifica, e a busca termina.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2+3 deixa X valendo a estrutura 2+3, sem calcular. X is 2+3 dá X = 5. fatorial(N, 120) dá erro de instanciação.

Saída do aritmetica.pl

```
_ = 2+3 → 2+3 = 2+3      _ is 2+3 → 5 is 2+3
5 =:= 2+3 → true         2+3 = 5 → false
7 mod 3 → 1              7 // 2 → 3             2 ^ 10 → 1024
fatorial(5,_) → 120      fatorial(0,_) → 1
fib(10,_) → 55           soma([1,2,3,4],_) → 10
tenta(fatorial(_,120))   → ERRO: instantiation_error
tenta(_ is _ + 1)        → ERRO: instantiation_error
```

4.1 Explique com exemplos a diferença entre =, is e =:=.

= unifica e não calcula. X = 2+3 deixa X valendo a estrutura 2+3, e 2+3 = 5 falha.
is calcula o lado direito e unifica o resultado com o esquerdo. X is 2+3 dá X = 5.
=:= calcula os dois lados e compara os valores. 5 =:= 2+3 é verdadeiro, e 2+3 =:= 1+4 também.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

O is e as comparações exigem que o lado direito esteja totalmente conhecido. Em fatorial(N,120), o teste N > 0 roda com N livre e causa o erro. A aritmética usa predicados primitivos que calculam em uma só direção, e o Prolog não resolve equações. Já o concatena só usa unificação de listas, então funciona em qualquer direção.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Igual: a estrutura é a mesma, com caso base 0 e passo n * fatorial(n-1). Diferente:

Em Haskell, fact é uma função que devolve o valor.
Em Prolog, o resultado é um argumento extra, e é preciso usar is e variáveis auxiliares (N1, F1).
Em Prolog, o teste N > 0 é necessário para a segunda cláusula não casar com 0.
A relação só funciona em um sentido.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro2(X) dá false. maximo2(5,3,M) dá duas respostas (5 e 3). maximo3(5,3,3) dá true, que é o erro.

Saída de controle.pl e mensagem de ancestral_ruim

```
tenta ana e ana
tenta ana e bia
tenta ana e caio
   resposta: mesmo_gosto(ana,caio)
tenta bia e ana
tenta bia e bia
tenta bia e caio
tenta caio e ana
   resposta: mesmo_gosto(caio,ana)
tenta caio e bia
tenta caio e caio
solteiro1(_) → osvaldo, paulo, jake
solteiro2(_) → false
maximo1(5,3,_) → 5      maximo2(5,3,_) → 5 e 3      maximo3(5,3,_) → 5
maximo1(5,3,3) → false  maximo3(5,3,3) → true
```

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

24 colorações com 3 cores, e com 2 cores nenhuma.

Saída do mapa.pl

```
[vermelho,verde,vermelho,verde,azul,vermelho,azul,verde]
[vermelho,verde,vermelho,azul,verde,vermelho,verde,azul]
24
false.
1728
```

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

As respostas foram (ana,caio) e (caio,ana), porque os dois gostam de café. Em “ana e ana” falhou a meta X \== Y, e o Prolog voltou à última escolha pendente, amigo(Y), para tentar a próxima amiga. Em “ana e bia” falhou gosta(bia, cafe), porque bia gosta de chá, e ele voltou para tentar outro Y.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

Em solteiro2, o \+ casado(X) roda com X ainda livre. Como casado(X) é provável (X = vern), a negação falha logo de início. A negação como falha só testa e não gera valores. Em solteiro1, o homem(X) gera o valor primeiro e depois a negação testa. O osvaldo aparece como solteiro porque não existe o fato casado(osvaldo) e vale o mundo fechado: não provar “casado” vira “não casado”. A base não diz nada sobre o estado civil dele.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O terceiro argumento já chega com valor 3. A cabeça maximo3(X,Y,X) tenta unificar X=5 com 3 e falha antes de chegar ao teste e ao corte. Então o corte nunca é executado, e o Prolog cai na 2ª cláusula, que não tem teste e aceita qualquer coisa. Correção, unificando depois do corte:

```
maximo3(X, Y, Z) :- X >= Y, !, Z = X.
maximo3(_, Y, Y).
```

Assim maximo3(5,3,3) dá false e maximo3(5,3,M) dá M = 5.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim tem recursão à esquerda. A primeira meta chama a própria regra com os mesmos argumentos, sem consumir nada, e o caso base fica na 2ª cláusula, que nunca é alcançada. A pilha estoura. ancestral_bom usa progenitor(X,Z) antes de recursar, então cada chamada anda um passo. As duas dizem o mesmo logicamente, mas o Prolog resolve em ordem fixa (de cima para baixo e da esquerda para a direita), então a ordem muda o resultado.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

Foram 24 colorações com 3 cores, 0 com 2 cores (false) e 1728 com 4 cores. Com 2 cores não existe solução, porque PR, SP e MS fazem fronteira entre si e formam um triângulo. O programa é declarativo porque só descreve o que é uma coloração válida (as restrições de fronteira). Quem faz a busca é o backtracking do Prolog, e o programa não tem laço nem algoritmo de busca.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma funcional, anexar é uma função que recebe duas listas e devolve a concatenada, em um só sentido. No lógico, concatena(A,B,C) é uma relação: serve para juntar, para separar (concatena(X,Y,[1,2,3]) dá 4 respostas) e para testar, com a mesma definição.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

1- A ordem de resolução importa. Em laco_infinito.pl, ancestral_ruim estoura a pilha e ancestral_bom não, embora sejam logicamente iguais.
2- A negação como falha com mundo fechado. Em controle.pl, solteiro2 dá false e solteiro1 dá osvaldo como solteiro só porque a base não tem casado(osvaldo).

Como alternativa, você pode citar a aritmética não relacional (fatorial(N,120) dá erro) ou o corte que muda o significado (maximo3(5,3,3) dá true).

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Eu escolheria Prolog para problemas de regras, relações e busca: sistemas especialistas, bancos de dados dedutivos, parentesco, análise de linguagem natural, restrições e escalonamento (como o mapa e o SEND+MORE). Ele descreve o problema e o backtracking busca a solução. Não escolheria para cálculo numérico pesado, interfaces gráficas, entrada e saída, estado mutável ou sistemas grandes com desempenho previsível. Aí o controle explícito do Java ajuda, e as limitações do Prolog (ordem, aritmética, corte) atrapalham.